Since your prompt trails off with *"from ....."*, here is the complete spectrum of **100 Vibe Coding Agent Guardrails**—ranging **from basic file hygiene and anti-regression rules all the way to autonomous terminal safety, security, and production readiness**.
You can drop these directly into your CLAUDE.md, .cursorrules, .windsurfrules, or custom agent system prompts.

I. Code Preservation & Anti-Regression (1–10)

The "Do Not Break Working Code" Rules

1. **No Unrequested Refactoring:** Never refactor, rename, or "clean up" working code outside the explicit scope of the current prompt.
2. **Ban Lazy Placeholders:** Never output // ... rest of code remains the same, // TODO: implement, or truncated blocks when writing files unless explicitly instructed to provide a diff.
3. **Preserve Existing Comments & Docs:** Do not strip out JSDoc blocks, architectural notes, or IMPORTANT: comments left by previous sessions.
4. **Surgical Edits Only:** Modify the minimum number of lines required to achieve the requested feature or bug fix.
5. **No Silent Feature Deletion:** If a change conflicts with an existing feature, pause and ask before deleting or disabling the existing logic.
6. **Lock Core Configs:** Never modify tsconfig.json, vite.config.ts, next.config.js, or bundler configs unless specifically tasked with a build/config issue.
7. **Respect Existing Patterns:** Match the repository's established naming conventions, folder structures, and state patterns instead of inventing a new paradigm mid-project.
8. **Backward-Compatible Signatures:** When updating a shared utility function, use optional parameters with defaults rather than breaking existing call sites.
9. **Verify Imports Before Removal:** Never delete an import statement without verifying it is completely unused across the entire file.
10. **Read Before Write:** Always read the target file and its immediate imports in full before proposing or applying edits.

II. Architecture, Modularity & File Hygiene (11–20)

Preventing the 3,000-Line Monolith Trap 11. **Hard File Line Limit:** No single source file may exceed 300–400 lines of code. Split into sub-modules, hooks, or services before hitting the ceiling. 12. **Single Responsibility Principle:** Separate UI rendering, business logic, data fetching, and type definitions into distinct files. 13. **Ban Circular Dependencies:** Never allow Module A to import from Module B if Module B (directly or indirectly) imports from Module A. 14. **Strict Layer Boundaries:** UI components must never directly query the database or import server-only SDKs; always route through an API or controller layer. 15. **No Orphaned Files:** When replacing or renaming a component/module, delete the old file and update all references in the same turn. 16. **Centralized Constants:** Never scatter magic numbers, hardcoded URLs, or timeout durations across components; place them in a dedicated constants/ file. 17. **Barrel File Discipline:** Avoid deep chains of index.ts barrel exports that break tree-shaking or cause bundler circularity. 18. **Colocate Feature Assets:** Keep feature-specific subcomponents, hooks, and tests inside their feature directory rather than dumping everything into a global utils/ folder. 19. **Pure Utility Functions:** Functions inside utils/ or helpers/ must be stateless, pure, and free of side effects. 20. **Explicit Public APIs:** Only export functions, classes, and types from a

module if they are actually consumed outside that file.

III. Dependency, Package & Environment Management (21–30)

Stopping Package Hallucinations & Version Hell 21. **Zero Hallucinated Packages:** Never import a library that isn't already listed in package.json / requirements.txt without installing it first. 22. **Check Native APIs First:** Prefer native runtime features (e.g., native fetch, crypto.randomUUID(), Intl, structuredClone) over installing third-party npm packages. 23. **Lockfile Integrity:** Never manually edit package-lock.json, pnpm-lock.yaml, or yarn.lock; only modify dependencies via the package manager CLI. 24. **Version Pinning Awareness:** Check the installed major version of frameworks (e.g., React 19 vs 18, Tailwind v4 vs v3, Express 5 vs 4) before writing syntax specific to a version. 25. **No Duplicate Purpose Libraries:** Do not install axios if fetch is used, or moment if date-fns is already installed, or a second icon library. 26. **Peer Dependency Check:** Verify peer dependency compatibility before adding UI primitives or plugin packages. 27. **Separate Dev vs. Prod Dependencies:** Always install linters, type definitions (@types/*), and test runners with -D / --save-dev. 28. **Verify Environment Variables at Boot:** Validate all required .env variables at application startup using a schema (e.g., Zod) and fail fast with a clear error message if any are missing. 29. **Maintain .env.example:** Every time a new environment variable is introduced in code, immediately append a dummy key to .env.example. 30. **Node/Python Engine Alignment:** Do not use runtime APIs unsupported by the version specified in .nvmrc or engines in package.json.

IV. TypeScript, Type Safety & Data Contracts (31–40)

Eliminating Runtime Type Crashes 31. **Strict any Ban:** Never use : any or as any to silence the compiler. Use unknown with type narrowing or proper generics. 32. **No @ts-ignore / @ts-expect-error:** Fix the underlying type mismatch instead of suppressing compiler errors. 33. **Shared Client-Server Schemas:** Define request/response contracts in a single shared schema (e.g., Zod or shared .types.ts) consumed by both frontend and backend. 34. **Explicit Return Types:** Always declare explicit return types on exported functions, API handlers, and custom hooks. 35. **Avoid Non-Null Assertions (!):** Never use user!.id unless preceded by a guaranteed check; use optional chaining (?.) and nullish coalescing (??) with safe fallbacks. 36. **Discriminated Unions for State:** Model async states as { status: 'loading' | 'success' | 'error' } unions rather than separate boolean flags that can contradict each other. 37. **Readonly Data Structures:** Use readonly arrays and properties for props and state payloads that should never be mutated in place. 38. **Runtime Boundary Validation:** Never trust external JSON (from APIs, localStorage, or webhooks) via as MyType; parse and validate it at runtime. 39. **Exhaustive switch Checks:** Include a default: const _exhaustiveCheck: never = value; branch in switch statements over union types. 40. **Strict Null Checks Compliance:** Always handle undefined and null explicitly when indexing arrays (noUncheckedIndexedAccess mindset).

V. Security, Secrets & Auth Boundaries (41–50)

Protecting Keys, Users & Infrastructure 41. **Zero Hardcoded Secrets:** Never write API keys, JWT secrets, database URIs, or private tokens directly into source files—even "temporarily for testing." 42. **Never Read or Print .env:** The agent must not dump .env contents to the console, logs, or external network requests. 43. **Server-Side Auth Enforcement:** Never rely on frontend route guards or hidden UI buttons for security; enforce authentication and role checks on every

backend endpoint. 44. **Parameterized Queries Only:** Never concatenate or interpolate user strings into SQL or NoSQL queries; strictly use prepared statements or ORM parameters. 45. **Sanitize Rendered HTML:** Never use dangerouslySetInnerHTML or v-html on user-generated content without passing it through DOMPurify first. 46. **Strict CORS Configuration:** Do not leave Access-Control-Allow-Origin: * on authenticated API endpoints in production configs. 47. **Rate Limiting on Sensitive Routes:** Always attach rate limiters to login, registration, password reset, and paid AI/LLM generation endpoints. 48. **Webhook Signature Verification:** Never process incoming payment or third-party webhooks (e.g., Stripe, Clerk, GitHub) without verifying the cryptographic signature header. 49. **IDOR Prevention (Ownership Check):** Always scope database updates and deletes by the authenticated userId alongside the resource id (e.g., WHERE id = $1 AND user_id = $2). 50. **Safe File Uploads:** Validate file MIME types, enforce strict size limits, and rename uploaded files to random UUIDs before storing.

VI. Frontend, UI/UX & State Management (51–60)

Preventing Janky, Unresponsive & Crashing UIs 51. **Mandatory Loading, Empty & Error States:** Every data-driven component must explicitly render a loading skeleton/spinner, an empty-state view, and a retryable error state. 52. **Prevent Infinite useEffect Loops:** Never put unstable object/array/function literals into effect dependency arrays without memoization, and never unconditionally set state inside an effect that depends on that state. 53. **Cleanup Side Effects:** Every addEventListener, setInterval, setTimeout, WebSocket, or Three.js/WebGL animation frame (requestAnimationFrame) must be cleaned up on component unmount. 54. **Optimistic UI Rollbacks:** If applying an optimistic UI update, always implement the catch rollback to restore previous state on network failure. 55. **Disable Submit During In-Flight Requests:** Every form button or action trigger must disable itself and show a pending state while its async request is running to prevent double-submissions. 56. **Responsive by Default:** Never use fixed pixel widths (e.g., w-[800px]) without responsive breakpoints or max-w-full constraints for mobile viewports. 57. **Design Token Consistency:** Use the project's defined theme colors, spacing scale, and typography classes rather than arbitrary hex codes (#ff3a2b). 58. **Stable List Keys:** Never use array index or Math.random() as a React/Vue list key if list items can be reordered, filtered, or deleted. 59. **Form Validation Before Network:** Validate user inputs client-side with clear field-level error messages before firing a backend request. 60. **Accessible Interactive Elements:** Never attach onClick to a or when a <button> or <a> is semantically required; preserve keyboard focus states.

VII. Backend, API & Database Safety (61–70)

Preventing Data Corruption & Server Lockups 61. **Idempotent Mutations:** Design critical write endpoints (payments, order creation, state transitions) to be idempotent using idempotency keys or unique constraints. 62. **Database Transactions for Multi-Step Writes:** Wrap any operation that modifies two or more tables (e.g., deducting credits + granting an item) inside an atomic database transaction (BEGIN / COMMIT / ROLLBACK). 63. **Enforce Pagination Limits:** Never write an unbounded SELECT * or .find() on user-facing endpoints; always enforce a limit (e.g., max 100 items) and cursor/offset pagination. 64. **Eliminate N+1 Queries:** Never execute a database query inside a for or .map() loop; use batch queries (WHERE id IN (...)) or joins. 65. **Destructive Migration Guardrail:** Never auto-run migrations that include DROP TABLE, DROP COLUMN, or TRUNCATE without explicit human confirmation. 66. **Explicit Timeouts on External Calls:** Every outbound fetch, database call, or third-party SDK request must include

an AbortSignal.timeout() or timeout config so hung services don't exhaust server connections. 67. **Standardized API Error Envelope:** Return consistent HTTP status codes and JSON error shapes ({ error: { code, message, details } }) across all routes—never return raw HTML stack traces. 68. **Connection Pool Singleton:** In serverless or hot-reloading dev environments, cache the database client on globalThis to prevent exhausting database connection limits. 69. **Background Heavy Tasks:** Never run video rendering, large file parsing, or batch LLM pipelines synchronously inside an HTTP request-response cycle; offload to a queue or background worker and return a 202 Accepted job ID. 70. **Graceful Shutdown Handling:** Handle SIGTERM and SIGINT signals to close active server requests and database pools cleanly.

VIII. Error Handling, Logging & Observability (71–80)

Banning Silent Failures 71. **Ban Empty Catch Blocks:** Never write catch (e) {} or catch (e) { console.log(e) } that swallows an error and lets execution continue with corrupted state. 72. **Contextual Error Wrapping:** When catching and re-throwing an error, attach context (e.g., which function failed, what resource ID was being processed). 73. **Zero PII in Logs:** Never log plaintext passwords, credit card numbers, full session tokens, or private user messages to stdout. 74. **Distinguish Operational vs. Programmer Errors:** Return friendly 4xx messages for invalid user input, and log 5xx alerts for unexpected bugs without leaking internal implementation details to the client. 75. **Global Unhandled Rejection Catch:** Ensure both the backend process and frontend app have top-level error boundaries (ErrorBoundary in React, unhandledRejection in Node) so a single component crash doesn't white-screen the entire app. 76. **Structured Logging:** Prefer structured logs (logger.error({ err, userId, action }, 'Failed to process')) over cryptic console.log("here 123") debugging statements. 77. **Remove Debug Artifacts Before Completion:** Strip temporary console.log, debugger, and test mock overrides before marking a task complete. 78. **Fail Loudly in Development:** In dev mode, surface missing configuration or schema mismatches immediately with actionable remediation steps. 79. **External API Rate-Limit Backoff:** Implement exponential backoff with jitter when calling rate-limited external APIs (e.g., OpenAI, Gemini, Stripe). 80. **Health Check Endpoint:** Maintain a lightweight /health or /api/health endpoint that verifies server readiness and database connectivity.

IX. Terminal, Git & Autonomous Execution Limits (81–90)

Keeping Autonomous Agents from Wrecking the Machine 81. **Ban Destructive Shell Commands:** Never execute rm -rf /, rm -rf *, git reset --hard, git push --force, or git clean -fd without explicit user approval. 82. **Scoped Directory Access:** Never read, modify, or execute files outside the current project workspace root directory. 83. **Non-Interactive CLI Flags Only:** Always pass non-interactive flags (-y, --yes, --no-input, CI=true) to terminal commands so the agent never hangs waiting for a stdin prompt. 84. **Port Conflict Awareness:** Before starting a dev server, check if the target port is already running an active instance rather than spawning duplicate zombie processes. 85. **Atomic Git Commits:** Commit working checkpoints before attempting risky multi-file refactors so rollback is a single command away. 86. **Never Commit Build Artifacts or Secrets:** Verify .gitignore excludes node_modules/, dist/, .env*, .DS_Store, and local SQLite/log files before staging (git add). 87. **Avoid git add . Blindly:** Stage specific modified files (git add <file1> <file2>) to avoid accidentally committing temporary scratch scripts or secrets. 88. **Kill Hung Subprocesses:** Enforce a hard execution timeout (e.g., 60 seconds)

on any test runner or build command executed by the agent. 89. **No Global System Mutations:** Never run sudo, modify /etc/hosts, alter global git configs, or install global npm/pip packages (npm i -g) without asking. 90. **Dry-Run Destructive Operations:** Use --dry-run flags first when running migrations, file batch deletions, or cloud deployment scripts.

X. Self-Verification, Reasoning & Loop Prevention (91–100)

Preventing Hallucination Spiral & "Death Loops" 91. **The 3-Strike Loop Breaker:** If the agent attempts to fix the same bug or compiler error 3 times and fails, it must **stop**, revert to the last working state, summarize what failed, and ask the user for guidance. 92. **Mandatory Typecheck/Build Gate:** Never declare a coding task "done" until tsc --noEmit (or the language's compiler/linter) passes with zero new errors. 93. **Fix Root Causes, Not Symptoms:** Do not patch a downstream crash with a dummy fallback if the bug is caused by an upstream function returning malformed data. 94. **Plan Before Multi-File Execution:** For any feature touching more than 3 files, output a brief bulleted execution plan (files to create/edit, data flow) before writing code. 95. **Clarify Ambiguous Requirements First:** If a prompt has critical ambiguity (e.g., unspecified auth provider, unclear pricing calculation), ask 1–3 concise clarifying questions before generating 1,000 lines of assumptions. 96. **No Fake Passing Tests:** Never modify a failing unit test to assert the broken output just to make the test suite turn green. 97. **Verify Imports & Exports Match:** After creating or editing a module, verify that named vs. default exports match every file importing that module. 98. **Browser/Runtime Smoke Check:** Verify that the application actually boots and renders without immediate runtime console errors after applying changes. 99. **Self-Diff Review:** Before finishing a turn, review git diff to ensure no unintended deletions, stray debug code, or formatting regressions were introduced. 100. **State Assumptions Explicitly:** Whenever making a minor architectural choice not specified by the user, state it briefly in a 1-line summary at the end of the response so the user stays in control of the codebase.

💡 Want this packaged into a ready-to-use file?

Tell me your primary stack or tool (e.g., **Cursor .cursorrules**, **Claude Code CLAUDE.md**, or an **autonomous coding agent system prompt**), and I can condense these 100 guardrails into a token-optimized, high-compliance instruction block tailored specifically to your workflow.